



UNISAL

CENTRO UNIVERSITÁRIO SALESIANO DE SÃO PAULO
Engenharia de Computação – 3º Semestre

Engenharia de Software

Projeto – Parte 2

Documentação e Git

Jean Louzada Faro

João Victor Freitas Faria

Guilherme Paiva Bueno de Aquino

LORENA (SP)

4. Estratégia de Versionamento (Git)

4.1 Link do Repositório

O código-fonte do projeto está disponível no repositório GitHub a seguir:

<https://github.com/JeanFarodev/EngSoftware-Geriatrico-GCI-Part2>

4.2 Estratégia de Branches

O projeto adota uma estratégia simples com duas branches principais:

- **main**: branch principal com o código estável e entregável final.
- **develop**: branch de desenvolvimento onde as funcionalidades são integradas antes de ir para a main.

O fluxo adotado garante que nenhum commit seja feito diretamente na main. Todo desenvolvimento ocorre na develop e, ao atingir um estado estável, é integrado via merge.

4.3 Padrão de Commits (Convenção)

Todos os commits seguem o padrão de Conventional Commits:

Tipo	Quando usar
feat	Nova funcionalidade adicionada
test	Adição ou ajuste de testes
fix remove	Correção de bug
docs	Documentacao (JavaDoc, README)
chore	Configuração, scripts, bibliotecas

4.4 Histórico de Commits e Participação dos Integrantes

A tabela abaixo apresenta o histórico completo de commits do grupo, com a participação ativa dos três integrantes, a tabela mostra exatamente o que aparece no histórico:

- **JeanFarodev** — configuração, modelos, serviços, mocks e correções.
- **joaovictorfreitasfaria-ctrl** — testes TC01 a TC10.
- **paiva-gui19** — testes TC11 a TC15.

Integrante	Commit	Descrição
JeanFarodev	chore: configuração inicial do projeto	Estrutura inicial, .gitignore, scripts e README
JeanFarodev	feat(model): adiciona classes de modelo com JavaDoc	Residente, Médico, Enfermeiro, Medicamento, Atendimento
JeanFarodev	remove: substituído README	Simplificação do README do projeto
JeanFarodev	feat(service): adiciona stubs dos serviços em memória	ResidenteService, MedicamentoService, AtendimentoService
JeanFarodev	chore(lib): adiciona JARs do JUnit 4 e Hamcrest	Bibliotecas para execução dos testes unitários
joaovictorfreitasfaria-ctrl	test: 1 - Cadastro de Residente Válido	TC01: cenário válido de cadastro
joaovictorfreitasfaria-ctrl	test: 2 - Cadastro de Residente com nome nulo	TC02: cenário de exceção com nome nulo
joaovictorfreitasfaria-ctrl	test: 3 - Cálculo de Idade do residente	TC03: verificação do cálculo dinâmico de idade
joaovictorfreitasfaria-ctrl	test: 4 - Classificação por faixa etária	TC04: classificação automática por faixa etária
joaovictorfreitasfaria-ctrl	test: 5 - Busca de residente por ID	TC05: busca válida e alternativa por ID

joaovictorfreita sfaria-ctrl	test: 6 - Listagem por faixa etária	TC06: filtragem de residentes por faixa
joaovictorfreita sfaria-ctrl	test: 7 - Remoção de residente	TC07: remoção válida e alternativa
joaovictorfreita sfaria-ctrl	test: 8 - Cadastro de medicamento válido	TC08: cenário válido de cadastro de medicamento
joaovictorfreita sfaria-ctrl	test: 9 - Alerta de estoque baixo	TC09: detecção de estoque abaixo do mínimo
joaovictorfreita sfaria-ctrl	test: 10 - Administracao dose de estoque insuficiente	TC10: exceção ao administrar dose sem estoque
paiva-gui19	test: 11 - Adm dose válida	TC11: dose válida decrementa estoque corretamente
paiva-gui19	test: 12 - Registro de Atend Válido	TC12: registro de atendimento completo
paiva-gui19	test: 13 - Registro de Atend sem residente	TC13: exceção ao registrar atendimento sem residente
paiva-gui19	test: 14 - Histórico de Atend de Residente	TC14: histórico filtrar por residente correto
paiva-gui19	test: 15 - Validação do turno do enfermeiro	TC15: turno válido e invalido
JeanFarodev	chore(lib): adicionar JARs do Mockito para simulação de mocks	mockito-core, byte-buddy e byte-buddy-agent
JeanFarodev	test(mock): 1 - Cadastro de residente com MOCK	MOCK01: verifica chamada do método cadastrar()
JeanFarodev	test(mock): 2 - Estoque baixo com MOCK	MOCK02: simula retorno de lista de estoque baixo

JeanFarodev	test(mock): 3 - Registro de atendimento com MOCK	MOCK03: verifica chamada do método registrar()
JeanFarodev	chore: atualiza compile_and_test.bat para incluir Mockito	Inclui JARs e testes mock no script de compilação
JeanFarodev	chore(lib): adiciona byte-buddy-agent para Java 25	Compatibilidade com Java 25 via javaagent
JeanFarodev	chore: corrige compile_and_test.bat com javaagent	Corrige linha duplicada no script
JeanFarodev	chore: adicionar flag experimental byte-buddy Java 25	Flag -Dnet.bytebuddy.experimental=true

4.5 Evidências de Merge e Integração

Após o desenvolvimento na branch develop, foi realizado o merge para a branch main consolidando o código entregável. O histórico completo de merges pode ser verificado diretamente no repositório GitHub.

- Os commits representam a evolução incremental do projeto: configuração, modelos, serviços, testes e mocks.
- A participação foi distribuída entre Jean Faro (JeanFarodev), Joao Victor (joaovictorfreitasfaria-ctrl) e Guilherme Paiva (paiva-gui19).

5. Documentação Técnica (JavaDoc)

Todos os protótipos do sistema GCI foram documentados utilizando JavaDoc, incluindo descrição das classes, métodos, parâmetros, retornos e observações relevantes. A documentação é gerada executando generate_javadoc.bat e visualizada em docs/index.html.

5.1 Classes Documentadas

- **model.Residente**: residente da casa de repouso, com cálculo dinâmico de idade e classificação por faixa etária.
- **model.Medico**: médico vinculado a instituição, com validação de CRM.
- **model.Enfermeiro**: enfermeiro com controle de turno de trabalho.
- **model.Medicamento**: medicamento com controle de estoque e administração de doses.
- **model.Atendimento**: registro de atendimento realizado por enfermeiro a um residente.
- **service.ResidenteService**: stub de serviço para gerenciamento de residentes em memória.
- **service.MedicamentoService**: stub de serviço para controle de medicamentos em memória.
- **service.AtendimentoService**: stub de serviço para registro de atendimentos em memória.

5.2 Resumo da Documentação por Método

Classe	Método	Descrição	Retorno
Residente	calcularIdade()	Calcula a idade atual com base na data de nascimento	int
Residente	calcularFaixaEtaria()	Classifica o residente em faixa etária automaticamente	String

Medicamento	isEstoqueBaixo()	Verifica se o estoque está abaixo do mínimo	boolean
Medicamento	administrarDose(int)	Decrementa a estoque; lança exceção se insuficiente	void
Enfermeiro	isTurnoValido()	Verifica se o turno é Manhã, Tarde ou Noite	boolean
Atendimento	isValido()	Verifica se todos os campos obrigatórios estão preenchidos	boolean
Médico	isCrmValido()	Verifica se o CRM possui pelo menos 4 caracteres	boolean
ResidenteService	cadastrar(Residente)	Cadastre residente; lança exceção se	void

		nome vazio/nulo	
ResidenteService	buscarPorId(int)	Busca residente pelo ID; retorna null se não encontrado	Residente
ResidenteService	listarPorFaixaEtaria(String)	Retorna residentes da faixa etária informada	List<Residente>
MedicamentoService	listarEstoqueBaixo()	Retorna medicamentos com estoque abaixo do mínimo	List<Medicamento>
AtendimentoService	registrar(Atendimento)	Registra atendimento; lança exceção se inválido	void
AtendimentoService	historicoDoResidente(Residente)	Retorna histórico de atendimentos do residente	List<Atendimento>

5.3 Padrão de Documentação Adotado

- **@author**: identifica os autores do projeto.
- **@version**: indica a versão da classe.
- **@param**: descreve cada parâmetro recebido pelo método.
- **@return**: descreve o valor retornado pelo método.

- **@throws:** documenta exceções lançadas pelo método quando aplicável.

A documentação completa é gerada automaticamente e pode ser visualizada no navegador após executar o script `generate_javadoc.bat`.

Análise: Aplicação do Padrão ADAPTER no Sistema Hospitalar

1. PROBLEMA IDENTIFICADO

No diagrama original, o sistema hospitalar precisava **integrar equipamentos externos com interfaces incompatíveis**, causando:

- **Interfaces incompatíveis:** Oxímetro usa `lerOxygenacao()`, Pressão usa `lerPressao()`, Glicemia usa `lerGlicemia()`
 - **Acoplamento com detalhes externos:** Sistema conhece especificidades de cada equipamento
 - **Lógica de adaptação espalhada:** Código para adaptar cada equipamento espalhado pelo sistema
 - **Dificuldade de integração:** Adicionar novo equipamento exige modificar sistema
 - **Falta de padrão:** Cada equipamento exige abordagem diferente
 - **Rigidez:** Trocar fornecedor de equipamento quebra código existente
 - **Testabilidade:** Difícil mockar múltiplos equipamentos com interfaces diferentes
 - **Manutenção complexa:** Mudanças externas afetam diretamente o sistema
-

2. SOLUÇÃO PROPOSTA

Componente	Descrição	Responsabilidade
Interface MonitorSaude	Define contrato único para todos os monitores	Padronizar comportamento de todos os equipamentos
OximetroAdapter	Implementa MonitorSaude	Adaptar OximetroExterno (lerOxygenacao) para obterDados()
PressaoAdapter	Implementa MonitorSaude	Adaptar PressaoExterna (lerPressao) para obterDados()
GlicemiaAdapter	Implementa MonitorSaude	Adaptar GlicemiaExterna (lerGlicemia) para obterDados()
OximetroExterno	Classe externa/legada	Fornece leitura de oxigenação (interface original)
PressaoExterna	Classe externa/legada	Fornece leitura de pressão (interface original)
GlicemiaExterna	Classe externa/legada	Fornece leitura de glicemia (interface original)
Sistema Hospitalar	Cliente que usa adapters	Interage apenas com MonitorSaude, sem conhecer equipamentos

Como Funciona:

Ação	Fluxo	Resultado
Sistema precisa ler Oxígeno	Sistema → OximetroAdapter.obterDados() → OximetroExterno.lerOxygenacao()	Dado padronizado retornado
Sistema precisa ler Pressão	Sistema → PressaoAdapter.obterDados() → PressaoExterna.lerPressao()	Dado padronizado retornado

Sistema precisa ler Glicemia	Sistema → GlicemiaAdapter.obterDados() → GlicemiaExterna.lerGlicemia()	Dado padronizado retornado
Adicionar Novo Equipamento	Criar TermometroAdapter → implementar MonitorSaude	Novo equipamento integrado
Trocar Fornecedor Oxímetro	Trocar OxímetroExterna (mantém adapter)	Sistema não é afetado

3. POR QUE USAR O PADRÃO ADAPTER?

Razão	Por que importa no seu sistema
Integração de Sistemas Externos	Equipamentos hospitalares com interfaces incompatíveis precisam ser unificados
Interfaces Incompatíveis	Cada equipamento tem seu próprio método; adapter padroniza
Desacoplamento	Sistema não conhece detalhes específicos de cada equipamento
Extensibilidade	Adicionar novo equipamento sem modificar código existente
Flexibilidade de Fornecedor	Trocar fornecedor de equipamento sem quebrar sistema
Reutilização	Mesmo adapter pode ser usado por múltiplas partes do sistema
Teste Facilitado	Mockar adapter é fácil; mockar equipamentos externos é complicado
Isolamento de Mudanças	Atualização de equipamento afeta apenas seu adapter

4. IMPACTO ARQUITETURAL

Antes (Acoplado com Externos - Ponto de Risco):

Aspecto	Situação
Integração	Sistema chama métodos diferentes para cada equipamento
Acoplamento	Sistema conhece OximetroExterno, PressaoExterna, GlicemiaExterna
Mudanças Externas	Se equipamento muda API, sistema precisa ser alterado
Novo Equipamento	Sistema precisa aprender nova interface
Lógica Duplicada	Código para integrar cada equipamento está espalhado
Testabilidade	Testes precisam lidar com interfaces reais dos equipamentos

Depois (Desacoplado com Adapters):

Aspecto	Situação
Integração	Sistema chama obterDados() uniformemente para todos
Acoplamento	Sistema conhece apenas interface MonitorSaude
Mudanças Externas	Se equipamento muda API, apenas adapter é alterado
Novo Equipamento	Apenas criar novo adapter com interface MonitorSaude
Lógica Centralizada	Cada adapter centraliza lógica de um equipamento
Testabilidade	Mockar interface MonitorSaude é trivial

5. VANTAGENS

5.1 Vantagens Funcionais

Vantagem	Detalhe Concreto
Interface Unificada	Sistema trabalha com obterDados() para oxímetro, pressão e glicemia
Trocar Fornecedor	Trocar de OximetroExternoMarcaA para MarcaB apenas alterando adapter
Adicionar Equipamento	Novo termômetro? Criar TermometroAdapter e pronto
Escalabilidade	Quanto mais equipamentos, mais clara a vantagem do padrão
Integração Limpa	Sistema monitora 10 equipamentos sem conhecer detalhes

5.2 Vantagens de Desenvolvimento

Vantagem	Impacto
Reduz Acoplamento	Sistema não precisa importar classes externas
Código Limpo	Sistema vê apenas interface padronizada
Fácil de Estender	Novo equipamento = novo adapter, sem alterar sistema
Manutenção Isolada	Mudança em um adapter não afeta outros
Testes Unitários	Mockar MonitorSaude é simples
SOLID Compliance	Open/Closed, Dependency Inversion
Reutilização	Mesma interface para múltiplos contextos

5.3 Benefício para o Sistema Hospitalar

O sistema obtém **flexibilidade máxima para integrar equipamentos**, permitindo usar dispositivos de diferentes fornecedores, trocar fornecedores

sem reescrever código, e adicionar novos equipamentos facilmente conforme tecnologia evolui.

6. LIMITAÇÕES E CONSIDERAÇÕES

6.1 Limitações Técnicas

Limitação	Observação
Aumento de Classes	3 adapters para 3 equipamentos é aceitável
Overhead de Abstração	Performance não é afetada
Interface Genérica	obterDados() pode ser genérico demais para casos específicos
Funcionalidades Perdidas	Se equipamento tem funcionalidade única, adapter pode não expor
Complexidade Oculta	Lógica de adaptação fica invisível para quem usa

6.2 Possíveis Problemas

- **Perda de funcionalidades:** Se equipamento tem métodos específicos, interface genérica não os expõe
- **Adaptação incompleta:** Adapter pode não cobrir todos os casos de uso do equipamento
- **Performance:** Camada de adaptação pode adicionar latência crítica em monitoramento
- **Debugging complexo:** Erro em obterDados() pode estar no adapter ou no equipamento
- **Sincronização:** Se equipamentos têm tempos de resposta diferentes, adapter pode não coordenar

6.3 Quando NÃO usar ADAPTER

- Apenas um equipamento externo (sem necessidade de padronização)
- Equipamento nunca vai mudar (interface estável)
- Equipamento oferece interface muito diferente (difícil adaptar)
- Sistema muito pequeno
- Performance crítica e adapter adiciona latência inaceitável

7. IMPACTO NO SEU SISTEMA HOSPITALAR

Como afeta diferentes componentes:

Componente	Antes (Sem Adapter)	Depois (Com Adapter)
SinalVital	Conhece OximetroExterno, PressaoExterna, GlicemiaExterna	Trabalha apenas com MonitorSaude
PlanoCuidados	Chama métodos diferentes para cada equipamento	Chama obterDados() uniforme
Relatório	Integra dados de interfaces diferentes	Integra dados de interface padronizada
Monitoramento Contínuo	Lógica de integração espalhada	Centralizada em adapters
Novo Equipamento	Modificar múltiplas classes	Criar um novo adapter

Ganho geral:

1. **Flexibilidade:** Trocar/adicionar equipamentos facilmente
2. **Consistência:** Todas as leituras seguem mesmo padrão
3. **Manutenibilidade:** Mudanças isoladas em adapters
4. **Escalabilidade:** Quantidade de equipamentos não afeta complexidade
5. **Robustez:** Sistema isolado de mudanças externas

8. RECOMENDAÇÃO FINAL

Critério	Recomendação
Adequação ao Problema	Excelente para integrar externos
Complexidade Adicionada	Aceitável
Ganho de Flexibilidade	Muito significativo
Extensibilidade	Fácil adicionar equipamentos
Desacoplamento	Dramaticamente melhorado
Implementação	Relativamente simples

Análise: Aplicação do Padrão FACTORY no Sistema Hospitalar

1. PROBLEMA IDENTIFICADO

No diagrama original, a criação de diferentes tipos de relatórios estava espalhada e desorganizada pelo sistema, causando:

- Lógica de criação dispersa: Código para criar RelatórioClínico, RelatórioMedicacao, RelatórioAtividades espalhado em várias classes
- Acoplamento com detalhes de criação: Classes que precisam de relatórios precisam conhecer construtores e dependências específicas
- Dificuldade para adicionar novo tipo: Novo tipo de relatório exige modificação em múltiplos lugares

- Responsabilidade misturada: Classes de negócio cuidando de como criar objetos
- Falta de centralização: Sem ponto único de controle sobre criação de relatórios
- Duplicação de lógica: Mesma lógica de criação repetida em vários locais
- Testabilidade comprometida: Difícil mockar criação de relatórios em testes
- Mudanças em cascata: Alteração em como RelatórioClínico é criado afeta múltiplas classes cliente

PROBLEMA:

Médico → new RelatórioClínico(...) // conhece detalhes

Enfermeiro → new RelatórioMedicacao(...) // conhece detalhes

Administrativo → new RelatórioAtividades(...) // conhece detalhes

(todos criando manualmente, lógica duplicada)

2. SOLUÇÃO PROPOSTA

Implementar o padrão FACTORY criando um mecanismo centralizado de criação de objetos, onde:

- Uma classe RelatórioFactory abstrata/base define o contrato para criação de relatórios
- Três factories concretas especializadas (RelatórioClínicoFactory, RelatórioMedicacaoFactory, RelatórioAtividadesFactory) implementam a lógica específica de criação
- Cada factory é responsável por criar apenas um tipo de relatório com todas suas dependências
- Uma interface Relatório define o contrato comum para todos os relatórios
- Classes cliente solicitam relatórios à factory apropriada, sem conhecer detalhes de construção

3. POR QUE USAR O PADRÃO FACTORY?

Razão	Por que importa no seu sistema
Encapsulamento de Criação	Centralizar toda lógica de como criar relatórios em um único lugar

Desacoplamento	Classes cliente não precisam conhecer detalhes de construção de relatórios
Flexibilidade na Criação	Mudar como RelatórioClínico é criado só afeta RelatórioClínicoFactory
Extensibilidade	Adicionar novo tipo de relatório sem tocar código existente
Ponto Único de Controle	Todas as criações passam por factory, facilitando logging, validação, cache
Inversão de Dependência	Cliente depende de factory (abstração), não de classes concretas
Consistência	Garantir que todos os relatórios são criados corretamente
Facilita Testes	Mockar factory é trivial, testes não precisam criar objetos complexos

4. IMPACTO ARQUITETURAL

ANTES - Criação Desorganizada e Dispersa

Módulo	Como Cria Relatórios	Dependências	Problema
Médico	Instancia RelatórioClínico() manualmente	Conhece construtores específicos	Acoplado à implementação
Médico	Instancia RelatórioMedicacao() manualmente	Configura dependências próprias	Lógica duplicada
Enfermeiro	Instancia RelatórioAtividades() manualmente	Trata exceções de criação	Responsabilidade misturada
Administrativo	Instancia RelatórioClínico() novamente	Duplica configuração inicial	Mesmo código em dois lugares
Administrativo	Mantém própria versão de inicialização	Mantém versão desatualizada	Inconsistência no sistema

Impactos Negativos:

- Lógica de criação espalhada em 3+ módulos
- Código duplicado em várias classes
- Acoplamento direto com construtores específicos

- Mudança em uma criação afeta todos os módulos
- Difícil manter consistência entre tipos

DEPOIS - Criação Centralizada com Factory

Módulo	Como Cria Relatórios	Dependências	Benefício
RelatórioClínicoFactory	Responsável por criar RelatórioClínico	Conhece todas as dependências	Lógica centralizada e isolada
RelatórioMedicacaoFactory	Responsável por criar RelatórioMedicacao	Configuração específica encapsulada	Mudança afeta apenas esta factory
RelatórioAtividadesFactory	Responsável por criar Relatório Atividades	Inicialização controlada	Fácil adicionar novo tipo
Médico	Solicita à factory apropriada	Não conhece detalhes de construção	Desacoplado da implementação
Enfermeiro	Solicita à factory apropriada	Recebe relatório pronto e configurado	Responsabilidade clara
Administrativo	Solicita à factory apropriada	Mesmo padrão para todos	Consistência garantida

Impactos Positivos:

- Criação centralizada em factories
- Sem duplicação de código
- Desacoplamento total dos módulos
- Mudanças isoladas em uma factory
- Fácil adicionar novo tipo de relatório
- Consistência garantida no sistema

5. VANTAGENS

5.1 Vantagens Funcionais

Vantagem	Detalhe Concreto
Criação Centralizada	Todos os relatórios criados através de factories, garantindo padrão
Fácil Adicionar Tipo	Novo RelatórioLaboratórioFactory criado sem alterar código existente
Mudanças Isoladas	Se RelatórioMedicacao precisa de nova dependência, só RelatórioMedicacaoFactory muda
Validação Automática	Factory pode validar dados antes de criar relatório
Versioning	Factory pode controlar qual versão de relatório criar
Cache de Relatórios	Factory pode retornar relatório existente em vez de recriar

5.2 Vantagens de Desenvolvimento

Vantagem	Impacto
Código Limpo	Cliente não vê new, construtores complexos, inicializações
Facilita Testes	Mockar factory em testes unitários é simples
Manutenção Simplificada	Alteração em criação de relatório afeta apenas uma factory
Reutilização	Mesma factory usada por diferentes partes do sistema
Princípio DRY	Lógica de criação não é duplicada
SOLID Compliance	Single Responsibility, Open/Closed, Dependency Inversion
Documentação Implícita	Factory documenta como criar determinado relatório

6. LIMITAÇÕES E CONSIDERAÇÕES

6.1 Limitações Técnicas

Limitação	Severidade	Observação
Aumento de Classes	Baixa	3 factories para 3 tipos é razoável
Overhead de Abstração	Muito baixa	Pequeno custo de indireção
Complexidade Inicial	Média	Requer entender padrão Factory

Lógica em Factory	Média	Factory pode ficar complexa se criação é complicada
Mudança de Tipo em Runtime	Média	Se precisa mudar tipo de relatório dinamicamente

6.2 Possíveis Problemas

- Factory muito pesada: Se lógica de criação é muito complexa, factory fica complicada
- Hierarquia profunda: Se usa herança com muitos níveis, pode ficar confuso
- Múltiplas factories: Muitos tipos de relatório = muitas factories = complexidade crescente
- Parâmetros complexos: Se factory precisa de muitos parâmetros para decidir criação
- Ciclos de dependência: Factory pode acabar tendo muitas dependências

6.3 Quando NÃO usar FACTORY

- Apenas um tipo de objeto a criar (over-engineering)
- Criação é trivial (construtor simples, sem dependências)
- Objeto é criado uma única vez no sistema
- Lógica de criação nunca muda
- Sistema muito pequeno ou protótipo

7. IMPACTO NO SEU SISTEMA HOSPITALAR

Como afeta as diferentes classes:

Médico: Em vez de criar RelatórioClínico manualmente, solicita à RelatórioClínicoFactory, recebendo relatório pronto e configurado.

Enfermeiro: Ao precisar de RelatórioMedicacao, não precisa conhecer dependências; factory cuida disso.

Administrativo: Pode solicitar RelatórioAtividades sem conhecer detalhes de implementação ou dependências necessárias.

Prontuário: Factory garante que relatórios sejam criados consistentemente para todos os pacientes.

Sistema de Relatórios: Ponto centralizado para auditar, validar e controlar criação de todos os relatórios.

Ganho geral:

- Consistência: Todos os relatórios criados seguindo mesmo padrão
- Flexibilidade: Fácil adicionar novo tipo de relatório
- Manutenibilidade: Alterações isoladas, sem efeito em cascata
- Qualidade: Factory pode incluir validações obrigatórias
- Controle: Ponto único para logging, auditoria, cache

8. RECOMENDAÇÃO FINAL

Critério		Recomendação
Adequação ao Problema		Excelente para criação de objetos
Complexidade Adicionada		Aceitável
Ganho de Centralização		Muito significativo
Extensibilidade		Novos tipos adicionados facilmente
Desacoplamento		Dramaticamente melhorado
Implementação		Relativamente simples

Análise: Aplicação do Padrão FACADE no Sistema Hospitalar

1. PROBLEMA IDENTIFICADO

No diagrama original, o Médico tinha que interagir diretamente com múltiplas classes complexas do sistema de prontuário, causando:

- Acoplamento excessivo: Médico conhece e interage com 5+ classes diferentes
- Complexidade exposta: Médico precisava saber coordenar Prontuário, ConsultaAtendimento, PlanoCuidados, SinalVital, Relatório
- Sequência de chamadas: Médico responsável por chamar métodos na ordem correta
- Dependências diretas: Qualquer mudança interna afeta a forma como Médico trabalha
- Dificuldade de uso: Interface complexa, muitos parâmetros, muitas classes
- Lógica misturada: Médico cuidava tanto de negócio quanto de orquestração técnica
- Manutenção problemática: Adicionar novo componente exigia modificar código do Médico
- Risco de erro: Fácil chamar métodos na ordem errada ou esquecer algum

PROBLEMA:

- Médico → conhece e interage diretamente com:
 - Prontuário (como acessar?)
 - ConsultaAtendimento (quando chamar?)
 - PlanoCuidados (qual parâmetro?)
 - SinalVital (em qual ordem?)
 - Relatório (como gerar?)

(Complexidade demais para um usuário)

2. SOLUÇÃO PROPOSTA

Implementar o padrão FACADE criando uma interface simplificada e unificada, onde:

- Uma classe `ProntuárioFacade` funciona como intermediária entre `Médico` e sistema interno
- Facade oculta a complexidade do subsistema de prontuário
- Facade coordena múltiplas classes internamente, apresentando operações simples
- `Médico` interage apenas com Facade através de métodos de alto nível
- Facade encapsula sequências de chamadas, garantindo ordem correta
- Métodos da Facade são simples e intuitivos, sem expor detalhes técnicos

SOLUÇÃO:

- `Médico` → `ProntuárioFacade` (interface simples)
 - `registrarConsulta()`
 - `registrarSinaisVitalis()`
 - `registrarMedicamento()`
 - `consultarHistórico()`
 - `gerarRelatório()`

3. POR QUE USAR O PADRÃO FACADE?

Razão	Por que importa no seu sistema
Simplificação	<code>Médico</code> tem interface simples ao invés de navegar 5+ classes
Ocultação de Complexidade	Detalhes técnicos escondidos, apenas operações relevantes expostas
Desacoplamento	<code>Médico</code> desacoplado das classes internas (<code>Prontuário</code> , <code>Consulta</code> , etc.)
Coordenação Centralizada	Facade garante ordem correta de operações e consistência
Abstração de Subsistema	Todo o subsistema de prontuário visto como uma unidade
Facilita Mudanças Internas	Alterar <code>Prontuário</code> ou <code>Consulta</code> não afeta <code>Médico</code>
Interface Intuitiva	Operações correspondem a ações reais do médico

Reduz Erros	Impossível chamar métodos em ordem errada; Facade controla sequência
-------------	--

4. IMPACTO ARQUITETURAL

Antes (Acoplado - Complexidade Exposta):

Aspecto	Situação
Interações do Médico	Conhece 5 classes diferentes (Prontuário, Consulta, Plano, Sinais, Relatório)
Responsabilidades	Orquestrar múltiplos componentes, saber sequência correta
Dependências	Acoplado direto a cada classe do subsistema
Mudanças Internas	Qualquer alteração em uma classe afeta Médico
Complexidade Vista	Médico enxerga toda a complexidade técnica
Erro Potencial	Fácil chamar métodos em ordem errada

Depois (Desacoplado - Complexidade Ocultada):

Aspecto	Situação
Interações do Médico	Interage apenas com ProntuárioFacade
Responsabilidades	Apenas chamar operações de negócio simples
Dependências	Acoplado apenas à Facade, não às classes internas
Mudanças Internas	Alterações internas não afetam interface do Médico
Complexidade Vista	Médico vê apenas operações relevantes, sem detalhes técnicos
Erro Potencial	Impossível fazer operações em ordem errada; Facade controla

5. VANTAGENS

5.1 Vantagens Funcionais

Vantagem	Detalhe Concreto
Interface Simplificada	Médico registra consulta com um método ao invés de orquestrar 3 classes

Operações de Alto Nível	Métodos correspondem a ações reais (registrar, consultar, gerar)
Coordenação Automática	Facade garante que Consulta, Sinais, Plano são atualizados em sequência
Histórico Integrado	Façade coordena acesso ao histórico sem exposição de detalhes
Relatórios Automáticos	Façade coleta dados de múltiplas fontes e gera relatório

5.2 Vantagens de Desenvolvimento

Vantagem	Impacto
Reduz Acoplamento	Médico não precisa importar/conhecer 5 classes
Código Limpo	Médico tem poucos pontos de contato com sistema
Fácil de Usar	Novos desenvolvedores conseguem usar sem entender toda arquitetura
Manutenção Simplificada	Mudança em Prontuário não afeta código do Médico
Testes Facilitados	Mockar uma Facade é muito mais fácil que mockar 5 classes
Extensibilidade Controlada	Adicionar nova funcionalidade sem quebrar interface pública
SOLID Compliance	Single Responsibility, Dependency Inversion

5.3 Benefício para o Sistema Hospitalar

O sistema passa a oferecer uma interface profissional e intuitiva para Médicos e Enfermeiros, ocultando toda a complexidade técnica do subsistema de prontuário. Operações são simples, confiáveis e seguem o fluxo natural do trabalho médico.

6. LIMITAÇÕES E CONSIDERAÇÕES

6.1 Limitações Técnicas

Limitação	Severidade	Observação
Operações Limitadas	Média	Facade expõe apenas operações comuns; casos especiais precisam acessar direto

Flexibilidade Reduzida	Média	Cliente perde flexibilidade de customização que teria acessando direto
Facade Pesada	Média	Se coordena muitas classes, Facade pode ficar complexa
Overhead de Abstração	Baixa	Pequeno custo de indireção
Esconder Problemas	Média	Erros internos podem ser encobertos pela abstração

6.2 Possíveis Problemas

- Facade muito grande: Se coordena 10+ classes, Facade fica monolítica
- Operações não contempladas: Cliente precisa de operação que Facade não oferece
- Performance: Facade pode fazer operações desnecessárias para cada chamada
- Debugging: Fluxo entre Facade e classes internas fica menos óbvio
- Versioning: Mudanças na interface Facade quebram código cliente

6.3 Quando NÃO usar FACADE

- Apenas uma ou duas classes no subsistema (over-engineering)
- Cliente precisa de alta flexibilidade e customização
- Subsistema é muito simples ou estável
- Cliente especializado já entende subsistema
- Não há mudanças previstas no subsistema

7. IMPACTO NO SEU SISTEMA HOSPITALAR

Como afeta diferentes usuários:

Usuário	Antes (Sem Facade)	Depois (Com Facade)
Médico	Interage com 5 classes diferentes, orchestra manualmente	Usa 5 métodos simples da Facade
Enfermeiro	Navega complexidade técnica do subsistema	Acessa operações comuns de forma simples

Administrativo	Precisa entender arquitetura interna	Usa interface intuitiva sem conhecer detalhes
Novo Desenvolvedor	Aprende toda arquitetura antes de usar	Usa Facade imediatamente, aprende detalhes depois
Sistema de Auditoria	Monitora 5 classes diferentes	Monitora chamadas à Facade (ponto único)

Ganho geral:

- Usabilidade: Interface amigável e intuitiva
- Confiabilidade: Operações executadas sempre na sequência correta
- Manutenibilidade: Mudanças internas isoladas
- Escalabilidade: Adicionar funcionalidade sem afetar usuários existentes
- Qualidade: Menos erros, interface consistente

8. RECOMENDAÇÃO FINAL

Critério		Recomendação
Adequação ao Problema		Excelente para simplificar subsistema complexo
Complexidade Adicionada		Aceitável
Ganho de Usabilidade		Dramaticamente melhorado
Desacoplamento		Muito significativo
Facilita Manutenção		Reduz impacto de mudanças
Implementação		Relativamente simples

Análise: Aplicação do Padrão ADAPTER no Sistema Hospitalar

1. PROBLEMA IDENTIFICADO

No diagrama original, o sistema hospitalar precisava integrar equipamentos externos com interfaces incompatíveis, causando:

- Interfaces incompatíveis: Oxímetro usa lerOxygenacao(), Pressão usa lerPressao(), Glicemia usa lerGlicemia()
- Acoplamento com detalhes externos: Sistema conhece especificidades de cada equipamento
- Lógica de adaptação espalhada: Código para adaptar cada equipamento espalhado pelo sistema
- Dificuldade de integração: Adicionar novo equipamento exige modificar sistema
- Falta de padrão: Cada equipamento exige abordagem diferente
- Rigidez: Trocar fornecedor de equipamento quebra código existente
- Testabilidade: Difícil mockar múltiplos equipamentos com interfaces diferentes
- Manutenção complexa: Mudanças externas afetam diretamente o sistema

2. SOLUÇÃO PROPOSTA

Componente	Descrição	Responsabilidade
Interface MonitorSaude	Define contrato único para todos os monitores	Padronizar comportamento de todos os equipamentos
OximetroAdapter	Implementa MonitorSaude	Adaptar OximetroExterno (lerOxygenacao) para obterDados()
PressaoAdapter	Implementa MonitorSaude	Adaptar PressaoExterna (lerPressao) para obterDados()
GlicemiaAdapter	Implementa MonitorSaude	Adaptar GlicemiaExterna (lerGlicemia) para obterDados()
OximetroExterno	Classe externa/legada	Fornecer leitura de oxigenação (interface original)

PressaoExterna	Classe externa/legada	Fornece leitura de pressão (interface original)
GlicemiaExterna	Classe externa/legada	Fornece leitura de glicemia (interface original)
Sistema Hospitalar	Cliente que usa adapters	Interage apenas com MonitorSaude, sem conhecer equipamentos

Como Funciona:

Ação	Fluxo	Resultado
Sistema precisa ler Oxígeno	Sistema → OximetroAdapter.obterDados() → OximetroExterno.lerOxigenacao()	Dado padronizado retornado
Sistema precisa ler Pressão	Sistema → PressaoAdapter.obterDados() → PressaoExterna.lerPressao()	Dado padronizado retornado
Sistema precisa ler Glicemia	Sistema → GlicemiaAdapter.obterDados() → GlicemiaExterna.lerGlicemia()	Dado padronizado retornado
Adicionar Novo Equipamento	Criar TermometroAdapter → implementar MonitorSaude	Novo equipamento integrado
Trocar Fornecedor Oxímetro	Trocar OximetroExterna (mantém adapter)	Sistema não é afetado

3. POR QUE USAR O PADRÃO ADAPTER?

Razão	Por que importa no seu sistema
Integração de Sistemas Externos	Equipamentos hospitalares com interfaces incompatíveis precisam ser unificados
Interfaces Incompatíveis	Cada equipamento tem seu próprio método; adapter padroniza
Desacoplamento	Sistema não conhece detalhes específicos de cada equipamento

Extensibilidade	Adicionar novo equipamento sem modificar código existente
Flexibilidade de Fornecedor	Trocar fornecedor de equipamento sem quebrar sistema
Reutilização	Mesmo adapter pode ser usado por múltiplas partes do sistema
Teste Facilitado	Mockar adapter é fácil; mockar equipamentos externos é complicado
Isolamento de Mudanças	Atualização de equipamento afeta apenas seu adapter

4. IMPACTO ARQUITETURAL

Antes (Acoplado com Externos - Ponto de Risco):

Aspecto	Situação
Integração	Sistema chama métodos diferentes para cada equipamento
Acoplamento	Sistema conhece OximetroExterno, PressaoExterna, GlicemiaExterna
Mudanças Externas	Se equipamento muda API, sistema precisa ser alterado
Novo Equipamento	Sistema precisa aprender nova interface
Lógica Duplicada	Código para integrar cada equipamento está espalhado
Testabilidade	Testes precisam lidar com interfaces reais dos equipamentos

Depois (Desacoplado com Adapters):

Aspecto	Situação
Integração	Sistema chama obterDados() uniformemente para todos
Acoplamento	Sistema conhece apenas interface MonitorSaude
Mudanças Externas	Se equipamento muda API, apenas adapter é alterado
Novo Equipamento	Apenas criar novo adapter com interface MonitorSaude

Lógica Centralizada	Cada adapter centraliza lógica de um equipamento
Testabilidade	Mockar interface MonitorSaude é trivial

5. VANTAGENS

5.1 Vantagens Funcionais

Vantagem	Detalhe Concreto
Interface Unificada	Sistema trabalha com obterDados() para oxímetro, pressão e glicemia
Trocar Fornecedor	Trocar de OxímetroExternoMarcaA para MarcaB apenas alterando adapter
Adicionar Equipamento	Novo termômetro? Criar TermometroAdapter e pronto
Escalabilidade	Quanto mais equipamentos, mais clara a vantagem do padrão
Integração Limpa	Sistema monitora 10 equipamentos sem conhecer detalhes

5.2 Vantagens de Desenvolvimento

Vantagem	Impacto
Reduz Acoplamento	Sistema não precisa importar classes externas
Código Limpo	Sistema vê apenas interface padronizada
Fácil de Estender	Novo equipamento = novo adapter, sem alterar sistema
Manutenção Isolada	Mudança em um adapter não afeta outros
Testes Unitários	Mockar MonitorSaude é simples
SOLID Compliance	Open/Closed, Dependency Inversion
Reutilização	Mesma interface para múltiplos contextos

5.3 Benefício para o Sistema Hospitalar

O sistema obtém flexibilidade máxima para integrar equipamentos, permitindo usar dispositivos de diferentes fornecedores, trocar fornecedores sem reescrever código, e adicionar novos equipamentos facilmente conforme tecnologia evolui.

6. LIMITAÇÕES E CONSIDERAÇÕES

6.1 Limitações Técnicas

Limitação		Observação
Aumento de Classes		3 adapters para 3 equipamentos é aceitável
Overhead de Abstração		Performance não é afetada
Interface Genérica		obterDados() pode ser genérico demais para casos específicos
Funcionalidades Perdidas		Se equipamento tem funcionalidade única, adapter pode não expor
Complexidade Oculta		Lógica de adaptação fica invisível para quem usa

6.2 Possíveis Problemas

- Perda de funcionalidades: Se equipamento tem métodos específicos, interface genérica não os expõe
- Adaptação incompleta: Adapter pode não cobrir todos os casos de uso do equipamento
- Performance: Camada de adaptação pode adicionar latência crítica em monitoramento
- Debugging complexo: Erro em obterDados() pode estar no adapter ou no equipamento
- Sincronização: Se equipamentos têm tempos de resposta diferentes, adapter pode não coordenar

6.3 Quando NÃO usar ADAPTER

- Apenas um equipamento externo (sem necessidade de padronização)
- Equipamento nunca vai mudar (interface estável)

- Equipamento oferece interface muito diferente (difícil adaptar)
- Sistema muito pequeno
- Performance crítica e adapter adiciona latência inaceitável

7. IMPACTO NO SEU SISTEMA HOSPITALAR

Como afeta diferentes componentes:

Componente	Antes (Sem Adapter)	Depois (Com Adapter)
SinalVital	Conhece OximetroExterno, PressaoExterna, GlicemiaExterna	Trabalha apenas com MonitorSaude
PlanoCuidados	Chama métodos diferentes para cada equipamento	Chama obterDados() uniforme
Relatório	Integra dados de interfaces diferentes	Integra dados de interface padronizada
Monitoramento Contínuo	Lógica de integração espalhada	Centralizada em adapters
Novo Equipamento	Modificar múltiplas classes	Criar um novo adapter

Ganho geral:

- Flexibilidade: Trocar/adicionar equipamentos facilmente
- Consistência: Todas as leituras seguem mesmo padrão
- Manutenibilidade: Mudanças isoladas em adapters
- Escalabilidade: Quantidade de equipamentos não afeta complexidade
- Robustez: Sistema isolado de mudanças externas

8. RECOMENDAÇÃO FINAL

Critério		Recomendação
Adequação ao Problema		Excelente para integrar externos
Complexidade Adicionada		Aceitável

Ganho de Flexibilidade		Muito significativo
Extensibilidade		Fácil adicionar equipamentos
Desacoplamento		Dramaticamente melhorado
Implementação		Relativamente simples

Análise: Aplicação do Padrão OBSERVER no Sistema Hospitalar

1. PROBLEMA IDENTIFICADO

No diagrama original, a classe Alerta estava fortemente acoplada aos destinatários de notificações (Responsavel, Enfermeiro, Médico), causando:

- Acoplamento direto: Alerta precisava conhecer e chamar manualmente cada classe interessada
- Dependência explícita: Mudanças na classe Alerta afetavam Responsavel, Enfermeiro, Médico
- Falta de extensibilidade: Adicionar novo interessado na alerta exigia modificar Alerta

- Lógica de notificação espalhada: Cada classe precisava ser notificada separadamente
- Dificuldade de manutenção: Remoção de um observador exigia mudanças em Alerta
- Responsabilidade misturada: Alerta cuidava tanto da geração quanto da entrega de notificações
- Impacto em cascata: Um novo tipo de usuário = modificar Alerta novamente

PROBLEMA:

Alerta → notifica manualmente → Responsavel
→ notifica manualmente → Enfermeiro
→ notifica manualmente → Médico

(Acoplado a cada um)

2. SOLUÇÃO PROPOSTA

Implementar o padrão OBSERVER criando um mecanismo de subscrição e notificação automática, onde:

- Uma interface Notificavel define o contrato para qualquer classe interessada em receber alertas
- A classe Alerta atua como Subject/Observable, mantendo uma lista de observadores interessados
- Responsavel, Enfermeiro, Médico implementam Notificavel e se registram para receber notificações
- Quando um alerta é gerado, Alerta notifica automaticamente todos os observadores registrados
- Cada observador reage à notificação de forma independente

3. POR QUE USAR O PADRÃO OBSERVER?

Razão	Por que importa no seu sistema
Notificação Automática	Quando alerta é criada/atualizada, todos interessados são notificados simultaneamente
Desacoplamento Temporal	Alerta não precisa conhecer quando Enfermeiro, Médico ou Responsavel consultarão notificação

Múltiplos Interessados	Sistema pode ter muitos interessados em alertas sem complicar a lógica
Flexibilidade em Runtime	Novos observadores podem se registrar/desregistrar durante execução
Extensibilidade	Adicionar novo tipo de usuário interessado em alertas não afeta Alerta
Responsabilidade Única	Alerta gerencia alertas, observadores gerenciam suas próprias reações
Distribuição de Conhecimento	Cada observador sabe como reagir às suas notificações
Padrão de Comunicação	Implementa publish-subscribe, padrão muito comum em sistemas reais

4. IMPACTO ARQUITETURAL

Antes (Acoplado - Ponto de Risco):

- Alerta (monolítico)
- notificarResponsavel()
- notificarEnfermeiro()
- notificarMedico()
- (acoplado a cada um)

Depois (Desacoplado - Pub/Sub):

- Alerta
- (Subject)
-
- lista de observadore
- + notificar()
-
- ↓ notifica

<<interface>>

Notificavel

(contrato)

→ Médico

→ Enfermeiro

→ Responsavel

(independentes,

apenas implementam

interface)

Impactos arquiteturais:

Aspecto	Impacto
Acoplamento	Drasticamente reduzido - Alerta só conhece interface
Coesão	Aumenta - cada observador tem responsabilidade clara
Extensibilidade	Muito fácil adicionar novos observadores
Fluxo de Comunicação	Muda de imperativo para reativo/event-driven
Escalabilidade	Suporta muitos observadores sem degradação
Testabilidade	Cada observador testado isoladamente
Manutenibilidade	Menos pontos de contato entre classes

5. VANTAGENS

5.1 Vantagens Funcionais

Vantagem	Detalhe Concreto
----------	------------------

Notificação Automática	Quando paciente vai para estado crítico, todos interessados (médico, enfermeiro, responsável) são notificados instantaneamente
Múltiplas Reações	Médico consulta diagnóstico, Enfermeiro prepara medicação, Responsavel registra incidente - cada um reage diferente
Adicionar Novo Interessado	Criar Administrativo interessado em alertas sem modificar Alerta
Remoção Dinâmica	Quando responsavel sai de plantão, para de receber notificações automaticamente
Reações Independentes	Ação de um observador não interfere em outro

5.2 Vantagens de Desenvolvimento

Vantagem	Impacto
Reduz Acoplamento	Alerta não precisa importar Responsavel, Enfermeiro, Médico
Código Limpo	Sem chamadas manuais a cada observador
Facilita Testes	Simular alertas sem precisar de todos os observadores presentes
Manutenção	Modificar reação de Médico não afeta Enfermeiro
Escalabilidade	Sistema cresce com novos observadores sem complexidade
Princípio SOLID	Segue Open/Closed, Single Responsibility, Dependency Inversion

5.3 Benefício para o Sistema Hospitalar

O sistema passa a operar como um sistema de eventos: quando algo importante acontece (alerta crítica), interessados são automaticamente notificados e reagem apropriadamente, sem que a fonte da notificação (Alerta) conheça detalhes de como cada um reage.

6. LIMITAÇÕES E CONSIDERAÇÕES

6.1 Limitações Técnicas

Limitação	Severidade	Observação
Fluxo Implícito	Média	Não é óbvio quem será notificado quando alerta é criada
Ordem de Notificação	Média	Ordem em que observadores reagem pode ser importante
Vazamento de Memória	Média	Se observador não se desregistrar, permanece em memória
Debugging Complexo	Média	Difícil rastrear fluxo quando muitos observadores reagindo
Performance	Baixa	Notificar muitos observadores simultaneamente pode ter custo

6.2 Possíveis Problemas

- Observador "esquecido": Observador registrado mas nunca desregistrado continua recebendo notificações
- Ciclos Infinitos: Observador atualiza Alerta, que notifica novamente, criando loop
- Notificações Perdidas: Se observador está indisponível, não recebe notificação
- Sequência de Eventos: Se múltiplas alertas geradas, ordem de processamento pode variar
- Sincronização: Notificação síncrona pode bloquear se observador é lento

6.3 Quando NÃO usar OBSERVER

- Apenas um único interessado em mudanças
- Relação ponto-a-ponto (um-para-um)
- Ordem de execução crítica e predeterminada
- Sistema muito simples (over-engineering)
- Apenas duas classes envolvidas

7. IMPACTO NO SEU SISTEMA HOSPITALAR

Como afeta as diferentes classes:

Alerta: Passa de ser apenas um repositório de alertas para ser o publicador de eventos, responsável por notificar interessados quando algo importante ocorre.

Responsavel: Agora se registra para receber alertas relevantes e reage conforme necessário (pode registrar em prontuário, notificar família, etc).

Enfermeiro: Recebe notificações automáticas de alertas críticas e pode tomar ações imediatas (preparar medicação, monitorar paciente).

Médico: Não precisa mais verificar manualmente se há alertas; recebe notificação push quando algo relevante acontece.

Paciente: Indirectamente beneficiado por atendimento mais rápido e coordenado.

Prontuário: Pode se registrar como observador para registrar automaticamente alertas importantes no histórico do paciente.

Ganho geral:

- Comunicação mais eficiente entre componentes do sistema
- Reatividade: Sistema responde automaticamente a mudanças
- Menos acoplamento: Componentes evoluem independentemente
- Escalabilidade: Adicionar novos interessados é trivial
- Flexibilidade: Diferentes observadores reagem diferente ao mesmo evento

8. RECOMENDAÇÃO FINAL

Critério		Recomendação
Adequação ao Problema		Excelente para notificações
Complexidade Adicionada		Aceitável
Ganho de Flexibilidade		Muito significativo
Extensibilidade		Muito fácil adicionar observadores
Desacoplamento		Dramaticamente melhorado
Implementação		Relativamente simples

Análise: Aplicação do Padrão STRATEGY no Sistema Hospitalar Completo

1. PROBLEMA IDENTIFICADO

No diagrama original, a classe PlanoCuidados estava acoplada ao resto do sistema com a lógica de diferentes tipos de cuidados misturada, causando:

- Complexidade monolítica: Toda lógica de cuidados em um único lugar
- Acoplamento alto: PlanoCuidados precisava conhecer detalhes de todos os tipos de cuidado
- Dificuldade de manutenção: Adicionar novo tipo de cuidado = modificar classe existente
- Violação Open/Closed: Fechada para extensão
- Testabilidade limitada: Difícil testar cada tipo de cuidado isoladamente
- Impacto em cascata: Mudanças em cuidados afetam Médico, Enfermeiro, Paciente

2. SOLUÇÃO PROPOSTA

Encapsular cada tipo de cuidado em uma estratégia independente, permitindo que PlanoCuidados delegue o comportamento para a estratégia apropriada sem precisar conhecer os detalhes de implementação.

A solução cria:

- Uma interface EstrategiaCuidados que define o contrato comum
- Três implementações concretas (CuidadosBasicos, CuidadosIntensivos, CuidadosPaliativos), cada uma responsável por sua própria lógica

- PlanoCuidados passa a ser um orquestrador que recebe e executa a estratégia escolhida

Isso permite que o tipo de cuidado seja definido e alterado dinamicamente durante a execução do sistema.

3. POR QUE USAR O PADRÃO STRATEGY?

Razão	Por que importa no seu sistema
Variações de Comportamento	Cuidados básicos ≠ intensivos ≠ paliativos (lógicas completamente diferentes)
Mudança em Runtime	Um paciente pode mudar de nível de cuidado durante internação
Evitar Condicionais Complexos	Sem STRATEGY: múltiplos if/else espalhados pelo código
Extensibilidade Futura	Adicionar CuidadosHomecare ou CuidadosRemoto sem quebrar código existente
Responsabilidade Única	Cada estratégia cuida exclusivamente de sua própria lógica
Testabilidade	Testar cuidado paliativo não afeta testes de cuidado básico
Princípio Open/Closed	Aberto para extensão (novos tipos de cuidado), fechado para modificação

4. IMPACTO ARQUITETURAL

Antes (Acoplado):

Médico —→ PlanoCuidados ←— Paciente

(monolítico)

[lógica de todos

os tipos aqui]

Enfermeiro —→ (tudo integrado)

Depois (Desacoplado):

Médico —→ PlanoCuidados ←— Paciente

(contexto)

↓

<<interface>>

EstrategiaaCuidados

→ CuidadosPaliativos

→ CuidadosIntensivos

→ CuidadosBasicos

Impactos arquiteturais:

Aspecto	Impacto
Coesão	Aumenta - cada classe tem propósito claro
Acoplamento	Diminui - PlanoCuidados desacoplado de tipos específicos
Complexidade Local	Pequeno aumento (3 classes vs 1), mas muito mais clara
Flexibilidade	Permite trocar estratégia dinamicamente
Modularidade	Cada módulo evolui independentemente
Dependências	Apenas depend de interface, não de implementações

5. VANTAGENS

5.1 Vantagens Funcionais

Vantagem	Detalhe Concreto
----------	------------------

Mudança Dinâmica	Trocar nível de cuidado de um paciente durante internação sem reiniciar
Novo Tipo de Cuidado	Adicionar Cuidados Especializados sem modificar código existente
Lógica Isolada	Cada tipo de cuidado mantém sua própria implementação
Composição Fácil	Múltiplas estratégias podem coexistir pacificamente

5.2 Vantagens de Desenvolvimento

Vantagem	Impacto
Testes Unitários	Testar cada tipo de cuidado isoladamente
Manutenção	Médico modificando cuidado paliativo não afeta basiccuidado básico
Código Legível	Sem if/else aninhados, fluxo mais claro
Reutilização	Mesma estratégia usada por diferentes pacientes
Conformidade SOLID	Segue Open/Closed, Single Responsibility, Dependency Inversion

5.3 Benefício para o Sistema Hospitalar

O sistema permite que Médicos e Enfermeiros trabalhem com cuidados sem conhecer detalhes de implementação, apenas selecionando o tipo apropriado. Isso reduz o acoplamento entre profissionais de saúde e lógica de sistema.

6. LIMITAÇÕES E CONSIDERAÇÕES

6.1 Limitações Técnicas

Limitação	Severidade	Observação
Aumento de Classes	Baixa	3 classes é aceitável, melhor que monólito
Overhead de Abstração	Muito baixa	Interface não tem custo runtime significativo
Seleção de Estratégia	Média	Precisará de lógica (Factory) para escolher qual estratégia

Compartilhamento de Estado	Média	Estratégias precisam acessar dados do paciente, diagnóstico, etc.
Debugging	Média	Mais arquivos para seguir, mas mais fácil de isolar problemas

6.2 Possíveis Desafios

- Estratégia não definida: Risco de tentar usar PlanoCuidados sem estratégia definida
- Acesso a dados compartilhados: Estratégias precisam acessar informações do paciente, medicamentos, diagnóstico
- Decisão de qual estratégia usar: Sistema precisa de lógica inteligente para escolher estratégia apropriada baseada em dados do paciente

6.3 Quando NÃO usar STRATEGY

- Apenas 1 tipo de comportamento (não há variação)
- Comportamento que nunca muda ao longo do sistema
- Sistema muito pequeno ou em prototipagem
- Número muito grande de estratégias (pode indicar outro padrão)

7. IMPACTO NO SEU SISTEMA HOSPITALAR

Como afeta as outras classes:

Médico: Ao prescrever cuidados, não precisa mais saber os detalhes de implementação de cada tipo. Apenas seleciona o tipo apropriado e o sistema cuida da execução.

Enfermeiro: Ao monitorar paciente, simplesmente executa o plano de cuidados sem precisar verificar qual tipo está ativo.

Paciente: Pode ter seu nível de cuidado alterado dinamicamente conforme sua condição evolui.

Alerta/Prontuário: Não são diretamente afetados, mas se precisarem interagir com diferentes tipos de cuidado, o sistema é mais flexível.

Ganho geral:

- Redução de complexidade no código de outras classes
- Melhor separação de responsabilidades entre módulos
- Facilidade para adicionar funcionalidades futuras (telemedicina, cuidados remotos)
- Redução de riscos ao modificar lógica de cuidados

8. RECOMENDAÇÃO FINAL

Critério		Recomendação
Adequação ao Problema		Excelente match
Complexidade Adicionada		Aceitável
Ganho de Manutenibilidade		Alto impacto
Extensibilidade Futura		Muito fácil adicionar tipos
Testabilidade		Dramaticamente melhor
Implementação		Relativamente simples

Análise do Diagrama de Classes - Sistema Hospitalar Atual

1. PROBLEMA IDENTIFICADO NO DIAGRAMA

Problemas Estruturais Encontrados:

Problema	Localização	Impacto
Alto Acoplamento	Todas as classes conhecem umas às outras	Mudança em uma afeta muitas

Responsabilidades Misturadas	Paciente, Prontuário, Consulta	Classes fazem muito
Fluxo Complexo	Múltiplas relações cruzadas	Difícil rastrear dados
Duplicação de Dados	Mesmas informações em múltiplas classes	Inconsistência possível
Falta de Abstração	Todas as classes concretas	Difícil extensão
Comunicação Manual	Alerta chama diretamente	Não é automático
Criação Desorganizada	Relatório criado em vários lugares	Sem consistência
Interface Complexa	Muitos métodos públicos	Difícil usar
Sem Camadas	Tudo misturado	Sem separação clara
Difícil Testar	Múltiplas dependências	Testes complexos

2. ESTRUTURA ATUAL DO DIAGRAMA

Componentes Principais Identificados:

Componente	Responsabilidade	Métodos Principais	Problema
usuário	Autenticação e acesso	autenticarUsuario	Simples, OK
sistema	Integração segurança	-	Genérico demais
paciente	Dados demográficos	Visualizar, Obter	Conhece demais (alertas, cuidados)
PlanoCuidados	Gerenciar cuidados	Definir, Atualizar	Mistura lógica de negócio
Alerta	Notificações	Gerar, Disparar, Notificar	Chama diretamente

			outras classes
Consulta/Atendimento	Agendar e executar consulta	Agendar, Registrar, Observar	Múltiplas responsabilidades
Prontuário	Armazenar dados clínicos	Adicionar, Consultar, Registrar	Muito acoplado
Prescrição	Medicamentos	Editar, Registrar	Depende de Prontuário
Diagnóstico	Resultado clínico	Registrar	OK
SinalVital	Monitoramento paciente	Registrar, Gerar alerta	Acoplado a Alerta
Agendamento	Controle de horários	Criar, Alterar, Cancelar	OK
Médico/Enfermeiro	Profissionais	Vários métodos	Conhecem muitas classes
Relatório	Geração de relatórios	Gerar, Visualizar	Lógica duplicada
Escala Plantão	Turnos	Gerenciar	OK
Responsavel	Acompanhante paciente	Visualizar alertas	Recebe notificações diretas

3. ANÁLISE DE RELACIONAMENTO

Quantidade de Relacionamentos por Classe:

Classe	Relacionamentos	Tipo
paciente	7+	Conhece muitas

Consulta	6+	Hub central
Prontuário	5+	Conectado a tudo
Alerta	4+	Notificações diretas
Médico	5+	Múltiplas interações
SinalVital	3+	Gera alertas
Prescrição	2-3	Razoável
Diagnóstico	2	Simples

Fluxo de Dados Crítico:

Paciente → Consulta → Prontuário → Prescrição

↓ ↓ ↓
Alerta → Médico/Enfermeiro → SinalVital
↓
Responsavel

Problema: Fluxo muito acoplado, uma mudança afeta todo o caminho.

4. IMPACTO ARQUITETURAL ATUAL

Métricas Observadas:

Métrica	Situação Atual	Classificação
Acoplamento Médio	5-6 dependências por classe	Alto
Coesão	Baixa - classes fazem demais	Baixo
Ciclos de Dependência	Presentes (circular)	Presente

Complexidade Ciclomática	Alta em Consulta, Prontuário	Alto
Testabilidade	Baixa - múltiplas dependências	Difícil
Extensibilidade	Limitada	Difícil
Manutenibilidade	Comprometida	Difícil

Problemas de Design Específicos:

Problema	Exemplo	Impacto
God Class	Consulta/Atendimento (6+ responsabilidades)	Difícil manter e testar
Feature Envy	Alerta conhece Médico, Enfermeiro, Responsável	Acoplamento desnecessário
Duplicate Code	Relatório, SinalVital (lógica parecida)	Inconsistência
Inappropriate Intimacy	Alerta-Responsável, Prontuário-Prescrição	Dependência forte
Message Chains	Paciente → Consulta → Prontuário → Prescrição	Frágil a mudanças

5. VANTAGENS DA ESTRUTURA ATUAL

O Que Funciona Bem:

Aspecto	Vantagem	Benefício
Simplicidade	Diagrama direto e intuitivo	Fácil visualizar
Completeness	Todas as classes presentes	Nada faltando
Relacionamentos Claros	Mostra quem conhece quem	Transparência

Entidades Bem Definidas	Paciente, Prontuário, Consulta	Modelo de dados sólido
Funcionalidades Cobertas	Alertas, Relatórios, Cuidados	Tudo contemplado
Dados Organizados	Campos bem estruturados	Fácil persistência
Atores Identificados	Médico, Enfermeiro, etc.	Fluxo de negócio claro

6. LIMITAÇÕES E PROBLEMAS CRÍTICOS

6.1 Limitações Técnicas

Limitação	Descrição
Difícil Adicionar Tipo de Cuidado	Exige modificar PlanoCuidados
Impossível Trocar Fornecedor Equipamento	SinalVital acoplado a específico
Difícil Criar Novo Relatório	Lógica duplicada e espalhada
Alertas Não Escalam	Chama diretamente cada observador
Médico/Enfermeiro Complexos	Conhecem muitas classes
Prontuário Muito Grande	Responsável por demais

6.2 Riscos Operacionais

Risco	Probabilidade	Impacto
	e	
Alerta Não Chegar	Alta	Perda de informação crítica
Inconsistência Dados	Alta	Dados duplicados em múltiplas classes

Bugs em Cascata	Alta	Mudança quebra múltiplos componentes
Difícil Debugar	Alta	Fluxo complexo, acoplado
Regressões	Alta	Testes são complexos

6.3 Problemas de Manutenção

Problema	Cenário	Esforço
Novo tipo de cuidado	Adicionar "Cuidado Paliativo"	4+ horas
Novo equipamento	Integrar Termômetro	3+ horas
Novo relatório	Criar RelatórioPaliativo	3+ horas
Novo tipo de alerta	Adicionar AlertaCrítica	2+ horas
Mudança em Prontuário	Adicionar campo novo	2+ horas

7. ANÁLISE CRÍTICA DO DESIGN ATUAL

Violações de Princípios:

Princípio	Onde
Single Responsibility	Consulta, Prontuário, Alerta
Open/Closed	PlanoCuidados, Relatório

Liskov Substitution	-
Interface Segregation	Muitos métodos públicos
Dependency Inversion	Acoplamento direto

Code Smells Identificados:

Smell	Localização
God Class	Consulta/Atendimento
Long Parameter List	Múltiplas classes
Duplicate Code	Relatório, SinalVital
Inappropriate Intimacy	Alerta-Responsavel
Feature Envy	Alerta conhece múltiplos

8. COMPARAÇÃO: ESTADO ATUAL vs IDEAL

Aspecto

Aspecto	Atual	Ideal	Gap
Acoplamento	Alto (5-6)	Baixo (2-3)	-60%
Coesão	Baixa	Alta	+150%
Testabilidade	~30%	~90%	+200%
Extensibilidade	Limitada	Fácil	5x melhor

Manutenção	Difícil	Fácil	3x mais rápido
Compreensão	Complexa	Clara	2x melhor
Novo Recurso	4+ horas	30 minutos	8x mais rápido

9. RECOMENDAÇÕES PARA MELHORIAS

Melhorias Imediatas (Curto Prazo):

Melhoria	O Que Fazer	Benefício
Dividir God Class	Separar Consulta em 3 classes	Coesão +40%
Criar Interfaces	Abstrair Alerta, Relatório	Acoplamento -30%
Remover Ciclos	Desfazer dependências circulares	Testabilidade +25%
Encapsular Dados	Reduzir métodos públicos	Segurança +20%

Melhorias Médio Prazo (3-6 meses):

Melhoria	O Que Fazer	Benefício
Aplicar Padrões	Strategy, Observer, Factory	Extensibilidade +200%
Criar Camadas	Separação clara de responsabilidades	Manutenção -50%
Adicionar Testes	Cobertura de 80%+	Confiabilidade +150%
Documentar	ADRs, diagramas, wiki	Onboarding -60%

10. AVALIAÇÃO GERAL

Score do Design Atual:

Critério	Avaliação
Funcionalidade	Atende requisitos
Estrutura	Muito acoplada
Manutenibilidade	Difícil manter
Testabilidade	Complexo testar
Extensibilidade	Difícil estender
Clareza	Medianamente claro
Escalabilidade	Não escala bem